

Intro to modeling in R

Tad Dallas

Contents

What are models?	1
What are the basic components of a model?	1
Model predictions	13
sessionInfo	19

Load the libraries first that we will use.

```
library(gbm)
```

```
## Loaded gbm 2.2.2
```

```
## This version of gbm is no longer under development. Consider transitioning to gbm3, https://github.com
```

```
library(rgbf)
```

```
library(maps)
```

```
##
```

```
## Attaching package: 'maps'
```

```
## The following object is masked from 'package:plyr':
```

```
##
```

```
## ozone
```

```
library(geodata)
```

What are models?

Models are representations of a system of interest, used for the purposes of inference or prediction. To say ‘model’ in ecology is a bit too broad to be useful, and the odd delineation between “modelers” and field researchers is a bit ridiculous. To start, we can delineate between statistical models and theoretical or phenomenological models. Statistical models (e.g., ANOVA, generalized linear models) are what we will cover here, and attempt to gain inference or prediction about some response y given some number of inputs $x_1 \dots x_n$. Phenomenological models are those that are constructed independent of theory but are an attempt to construct a system which can explain some ecological process, often in a dynamic way (think of the SIR model in epidemiology). Theoretical models are those that directly arise from first principles independent of empirical data. The main distinction here is between statistical models and “other” models, as both are neat, but we’ll focus on simple statistical models here.

When you create a model of a system, you have two things you don’t understand; the system and the model.

What are the basic components of a model?

The response variable should be the thing that you want to predict, or the thing whose variability you want to understand (also sometimes referred to as the dependent variable). Often, it is something that you think is the effect produced by some other cause. For example, in examining the relationship between gas usage

and outdoor temperature, it seems clear that gas usage should be the response: temperature is a major determinant of gas usage. But suppose that the modeler wanted to be able to measure outdoor temperature from the amount of gas used. Then it would make sense to take temperature as the response variable.

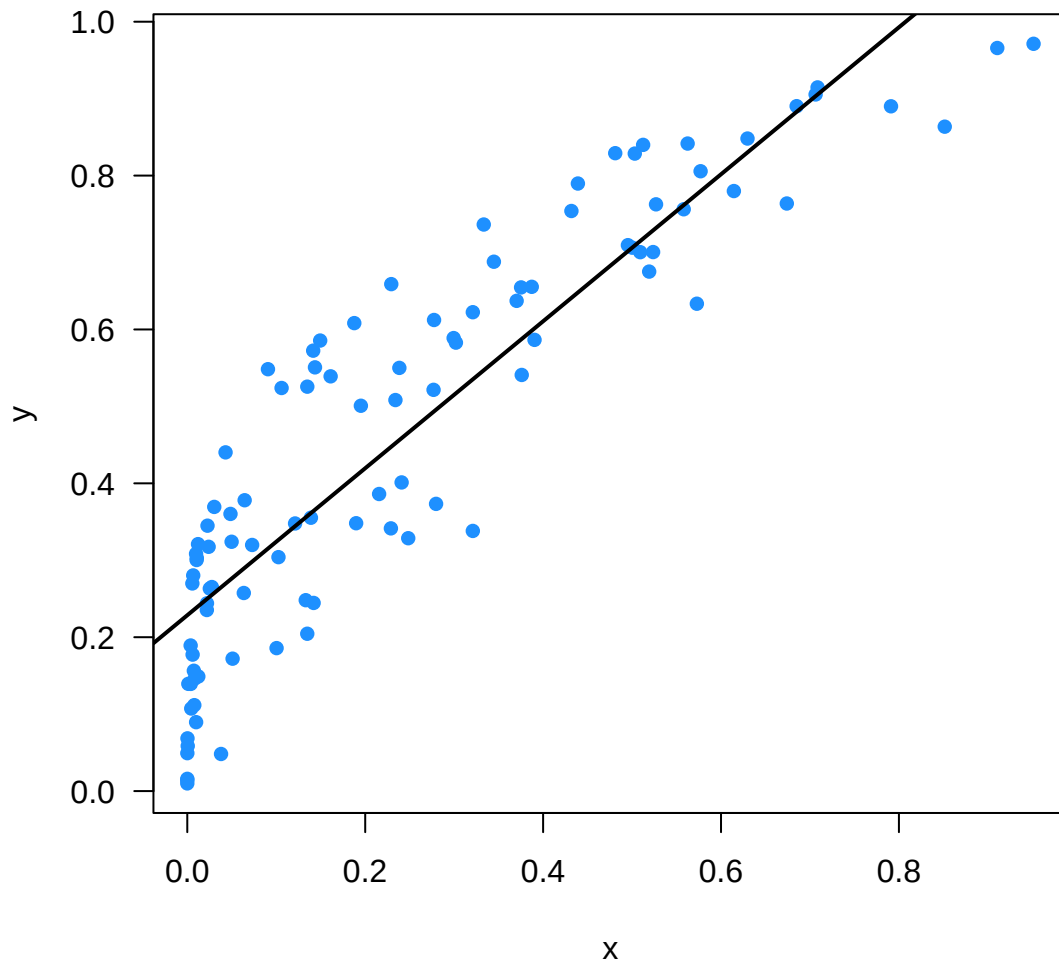
The explanatory variables are used to explain or predict the response variable (also sometimes referred to as independent variables).

A simple example

```
y <- runif(100)
x <- y ** runif(100, 1,4)

plot(y=y,x=x, pch=16,
     ylab='y', xlab='x',
     col='dodgerblue', las=1)

m1 <- lm(y~x)
abline(m1, lwd=2)
```



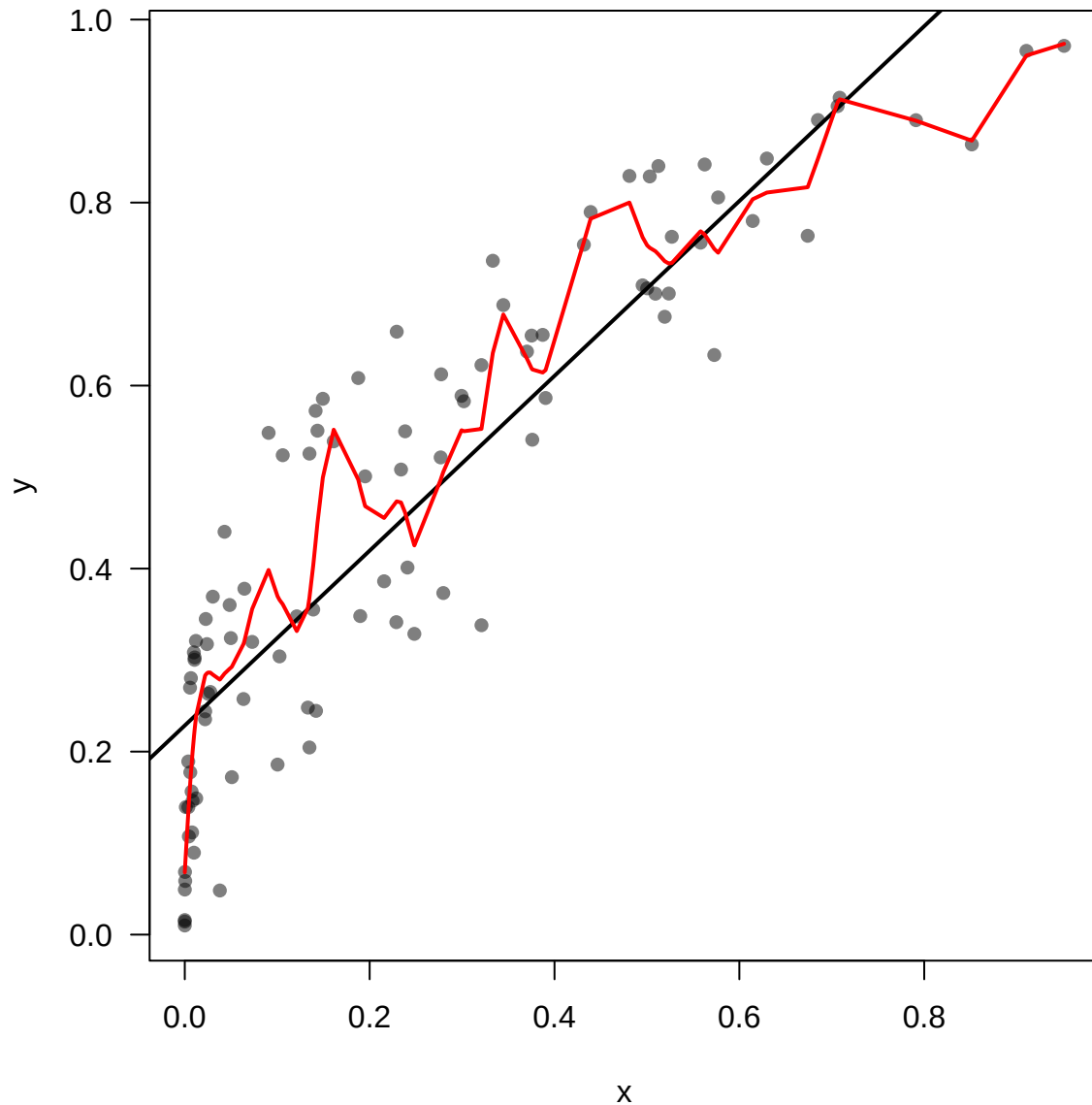
But to minimize these residuals (or the sum of squares of the residuals), we don't necessarily *need* a model, right?

```

par(mar=c(4,4,0.5,0.5))
plot(y=y,x=x, pch=16,
     ylab='y', xlab='x',
     col=adjustcolor(1,0.5), las=1)
abline(m1, lwd=2)

lines(smooth.spline(y=y,x=x), col='red', lwd=2)

```



And if we accept that we need a model, what's stopping us from making the model incredibly complex to capture all the possible variation in the data?

```

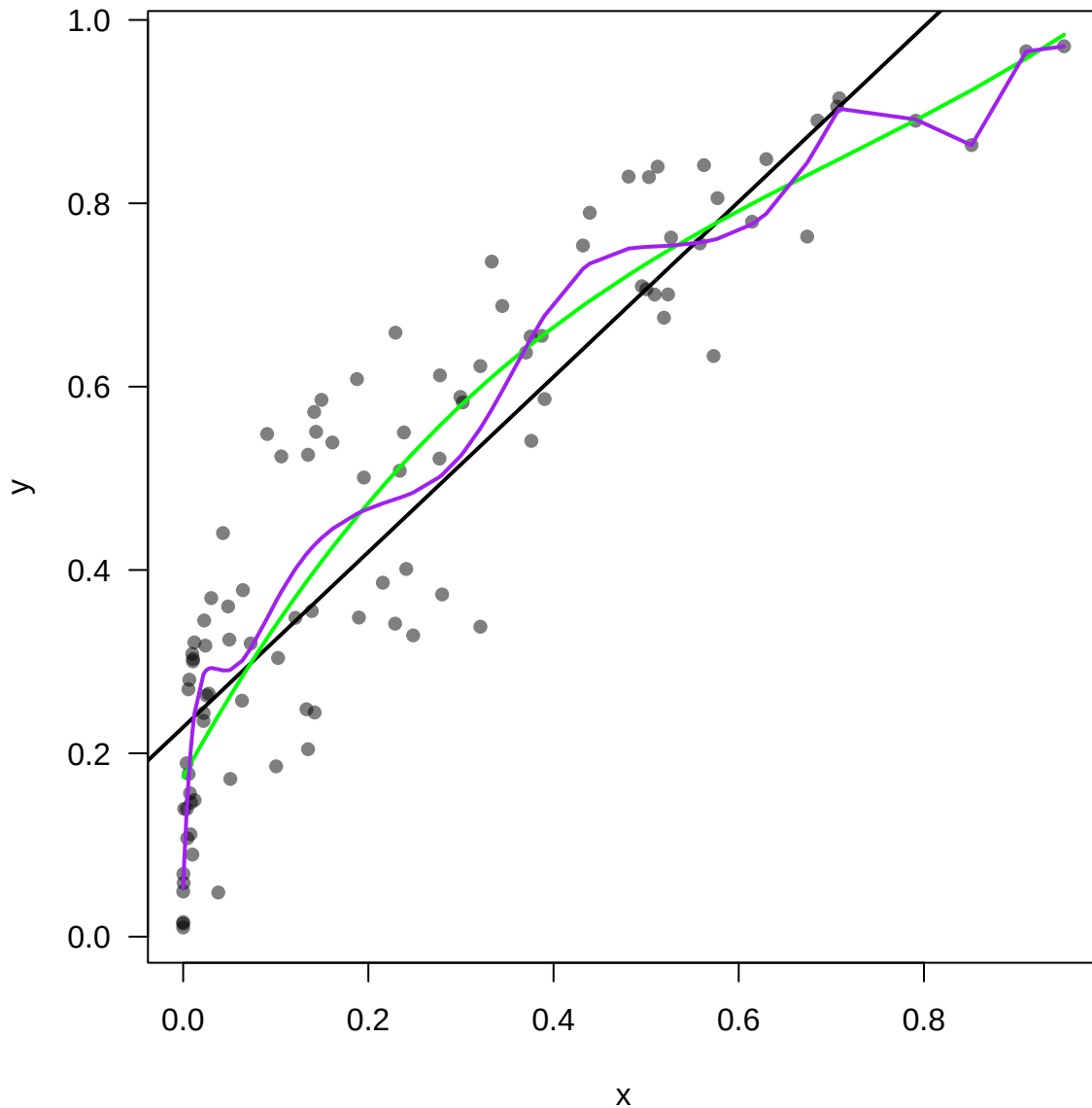
par(mar=c(4,4,0.5,0.5))
plot(y=y,x=x, pch=16,
     ylab='y', xlab='x',
     col=adjustcolor(1,0.5), las=1)
abline(m1, lwd=2)

m3 <- lm(y~poly(x,3))

```

```
lines(x=sort(x), fitted(m3)[order(x)], col='green', type='l', lwd=2)

m4 <- lm(y~poly(x,15))
lines(x=sort(x), fitted(m4)[order(x)], col='purple', type='l', lwd=2)
```



The model would quickly become uninformative. That is, the model could fit every data point perfectly, but tell us nothing about the general relationship between the two variables. This makes it impossible to use the model to predict. Adding in this level of complexity essentially minimizes the error to the fit, but in turn basically maximizes error when extrapolating to new data. This model would be overfit, and the process of adding in variables that minimize model fit error while making the model useless to other datasets, is called overfitting. Not ideal.

Breaking the assumptions of the linear model

Linear models have the standard assumptions of independent and identically distributed (i.i.d.) normal random variables.

More explicitly, these standard assumptions are:

- independence: samples are not autocorrelated or suffer from some other non-independence
- identically distributed: samples come from the same probability distribution
- normality: samples come from a normal probability distribution (error distribution is gaussian)

We'll focus on the last assumption here, since it's the easiest and most common to have to work around. For instance, if the response variable is a binary outcome (e.g., dead or alive, present or absent).

To handle these issues, a common approach is to use generalized linear models, which attempt to account for response data with non-normal distribution. This is done by using a link function to provide an appropriate *link* between the random error model (the linear model would assume normally distributed errors) and the predictors.

Probability distribution	Typical uses	Link name
Normal	linear-response data	Identity
Poisson	count of occurrences in fixed amount of time/space	Log
Bernoulli	outcome of single yes/no occurrence	Logit
Binomial	count of # of "yes" occurrences out of N yes/no occurrences	Logit

An example

Logistic regression

Random component: The distribution of Y is assumed to be Binomial(n, π), where π is a probability of "success".

Systematic component: X's are explanatory variables (can be continuous, discrete, or both) and are linear in the parameters, e.g., $\beta_0 + \beta_1 x_i + \dots$.

Again, transformation of the X's themselves are allowed like in linear regression; this holds for any GLM.

Link function: Logit link

$$\nu = \text{logit}(\nu) = \log\left(\frac{\pi}{1 - \pi}\right)$$

```
y2 <- rbinom(100, 1, 0.5)
m5 <- glm(y2 ~ x, family = binomial)
summary(m5)

##
## Call:
## glm(formula = y2 ~ x, family = binomial)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.1524      0.2842  -0.536   0.592
## x           0.7850      0.8262   0.950   0.342
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 138.59  on 99  degrees of freedom
## Residual deviance: 137.67  on 98  degrees of freedom
## AIC: 141.67
##
```

```
## Number of Fisher Scoring iterations: 4
```

glm is incredibly flexible and is the basis for more complicated models like hierarchical models, ridge regressions, generalized additive models, etc. etc. Some other forms of generalized regression are:

- Generalised additive models, e.g. mgcv::gam(), extend generalised linear models to incorporate arbitrary smooth functions. That means you can write a formula like $y \sim s(x)$ which becomes an equation like $y = f(x)$ and let gam() estimate what that function is (subject to some smoothness constraints to make the problem tractable).
- Generalized linear mixed effects models, e.g., lme4::glmer extend generalized linear models to incorporate both fixed and random effects.
- Trees, e.g. rpart::rpart(), attack the problem in a completely different way than linear models. They fit a piece-wise constant model, splitting the data into progressively smaller and smaller pieces. Trees aren't terribly effective by themselves, but they are very powerful when used in aggregate by models like random forests (e.g. randomForest::randomForest()) or gradient boosting machines (e.g. xgboost::xgboost.)

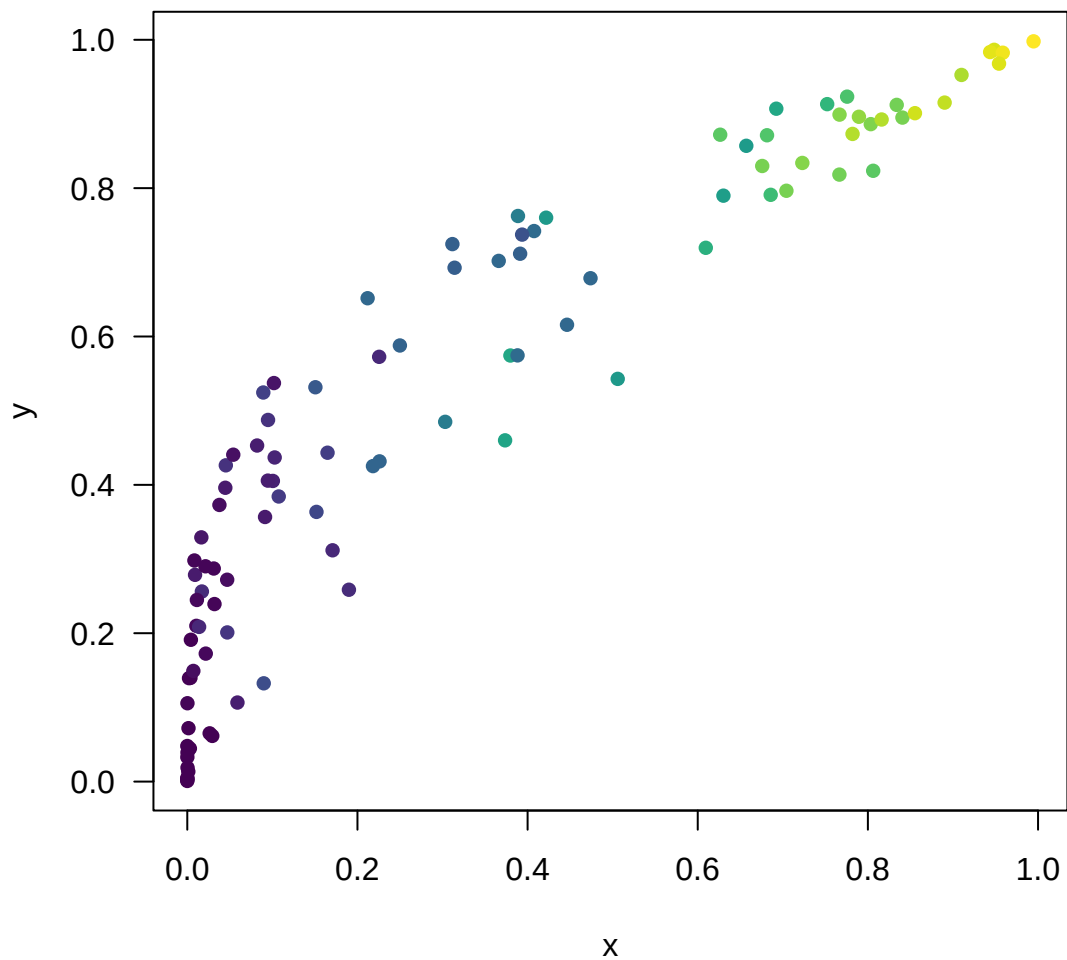
Feature creation

So we've gone over modeling largely in terms of linear models of some arbitrary number of predictors, which our examples only included a single predictor. When designing a model, the researcher may want to incorporate known variable associations or interactions to estimate the support for a hypothesized effect given the data and model. This can be done by using the * operator in the formulate interface.

```
y <- runif(100)
x1 <- y ** runif(100, 1,4)
x2 <- x1 ** runif(100, 0.5,1.5)
dat <- data.frame(y,x1,x2)

m6 <- lm(y~x1+x2, data=dat)
m7 <- lm(y~x1*x2, data=dat)

plot(y=y,x=x1, pch=16,
     ylab='y', xlab='x',
     col=viridis::viridis(100)[as.numeric(cut(x2,100))], las=1)
```



```
summary(m6)
```

```
##
## Call:
## lm(formula = y ~ x1 + x2, data = dat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.218455 -0.079515 -0.000387  0.086634  0.240474
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   0.21931    0.01764  12.430 < 2e-16 ***
## x1             0.83238    0.18342   4.538 1.63e-05 ***
## x2             0.04559    0.18372   0.248  0.805
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1229 on 97 degrees of freedom
## Multiple R-squared:  0.8502, Adjusted R-squared:  0.8472
## F-statistic: 275.4 on 2 and 97 DF,  p-value: < 2.2e-16
```

```
summary(m7)
```

```
##
## Call:
## lm(formula = y ~ x1 * x2, data = dat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.215095 -0.078634  0.006115  0.068767  0.232869
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   0.16001    0.01769   9.046 1.66e-14 ***
## x1             1.28156    0.17107   7.492 3.33e-11 ***
## x2             0.34485    0.16274   2.119  0.0367 *
## x1:x2        -0.86908    0.13869  -6.266 1.05e-08 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1041 on 96 degrees of freedom
## Multiple R-squared:  0.8937, Adjusted R-squared:  0.8904
## F-statistic: 269.1 on 3 and 96 DF,  p-value: < 2.2e-16
```

What if we don't necessary know the direct interaction, but still have some idea that a combination of variables is likely associated with some response y ? We can do what is called feature creation (feature is just another word people use to mean variable, most often applied to creating informative predictors). For instance, if we were trying to predict the time until a bottled beer were to go skunky, we could include a measure of bottle opacity (controls the amount of light that can get in and affect the beer), and aspects of bottle dimensions (height, width). What may be important is some combination of these variables, perhaps starting to get at volumn differences or surface area to volume ratio which may influence light attenuation. So we can create a variable that represents total beer volume.

Another example that is often used is to take many many predictors which may be highly collinear and to engineer features from this set. This is often done through what is referred to as 'feature reduction', a form of feature creation that attempts to make a small set of often-independent predictor variables from a large set of collinear variables. Think of climate data, where we have temperature on each day of the year as a set of 364 predictor variables explaining ice cream sales (our response). This model would likely not fit so well with so many predictors. But we could compress the information using feature reduction approaches like principal component analysis (PCA) or just some ad hoc idea about temperature (monthly mean temperature is the most important, reducing our feature set to 12 variables).

In the previous example, we had two continuous predictors (x_1 and x_2), but variables can also be categorical or factor variables.

```
y.1 <- runif(1000)
x.1 <- y * runif(1000, 0.6,1.4) + runif(1000)
x.2 <- c('a', 'b')[1+rbinom(length(x.1), 1, prob=x.1/max(x.1))]
dat2 <- data.frame(y.1,x.1,x.2)

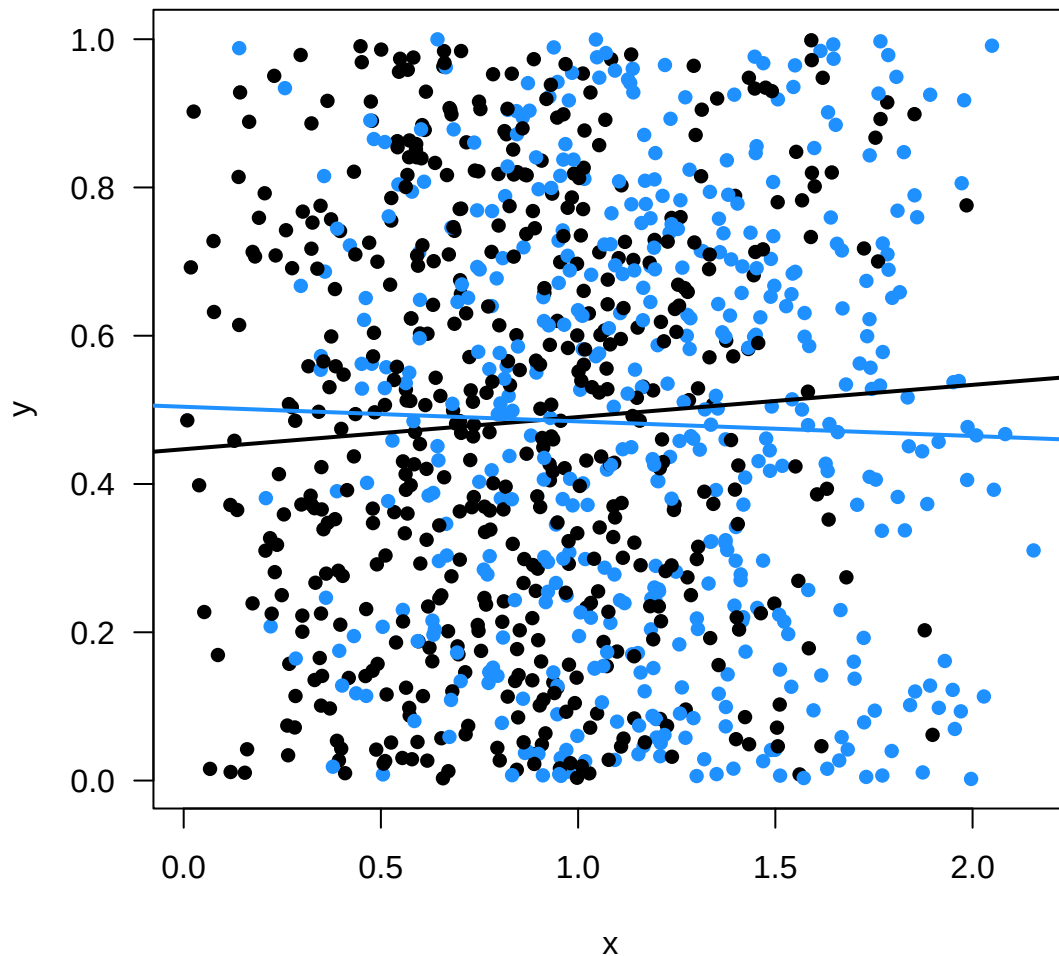
m8 <- lm(y.1~x.1+x.2, data=dat2)

plot(y=y.1, x=x.1, pch=16,
     ylab='y', xlab='x',
     col=c(1, 'dodgerblue')[as.factor(x.2)],
     las=1)
```



```
summary(m8)
```

```
##
## Call:
## lm(formula = y.1 ~ x.1 + x.2, data = dat2)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.48845 -0.25152  0.00695  0.23739  0.52503
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.473248   0.022303  21.219  <2e-16 ***
## x.1          0.011706   0.022237   0.526    0.599
## x.2b        -0.006012   0.019711  -0.305    0.760
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2868 on 997 degrees of freedom
## Multiple R-squared:  0.0002898, Adjusted R-squared:  -0.001716
## F-statistic: 0.1445 on 2 and 997 DF,  p-value: 0.8654
m8_a <- lm(y.1~x.1, data=subset(dat2, dat2$x.2=='a'))
m8_b <- lm(y.1~x.1, data=subset(dat2, dat2$x.2=='b'))
abline(m8_a, lwd=2, col=1)
abline(m8_b, lwd=2, col='dodgerblue')
```



Simpson's paradox

Here, we run into a cautionary tale of variable inclusion and when model building takes a bit more knowledge of a system. In this simple example, borrowing most of the code from this user (https://commons.wikimedia.org/wiki/File:Simpson%27s_paradox_continuous.svg), we see that not including a categorical variable can result in a complete misinterpretation of the overall pattern. That is, both categorical groups have a positive relationship between y and x , but ignoring the categorical variable results in an overall negative relationship between y and x . This is referred to as “Simpson’s paradox”, and there are plenty of ecological systems where this is definitely a thing.

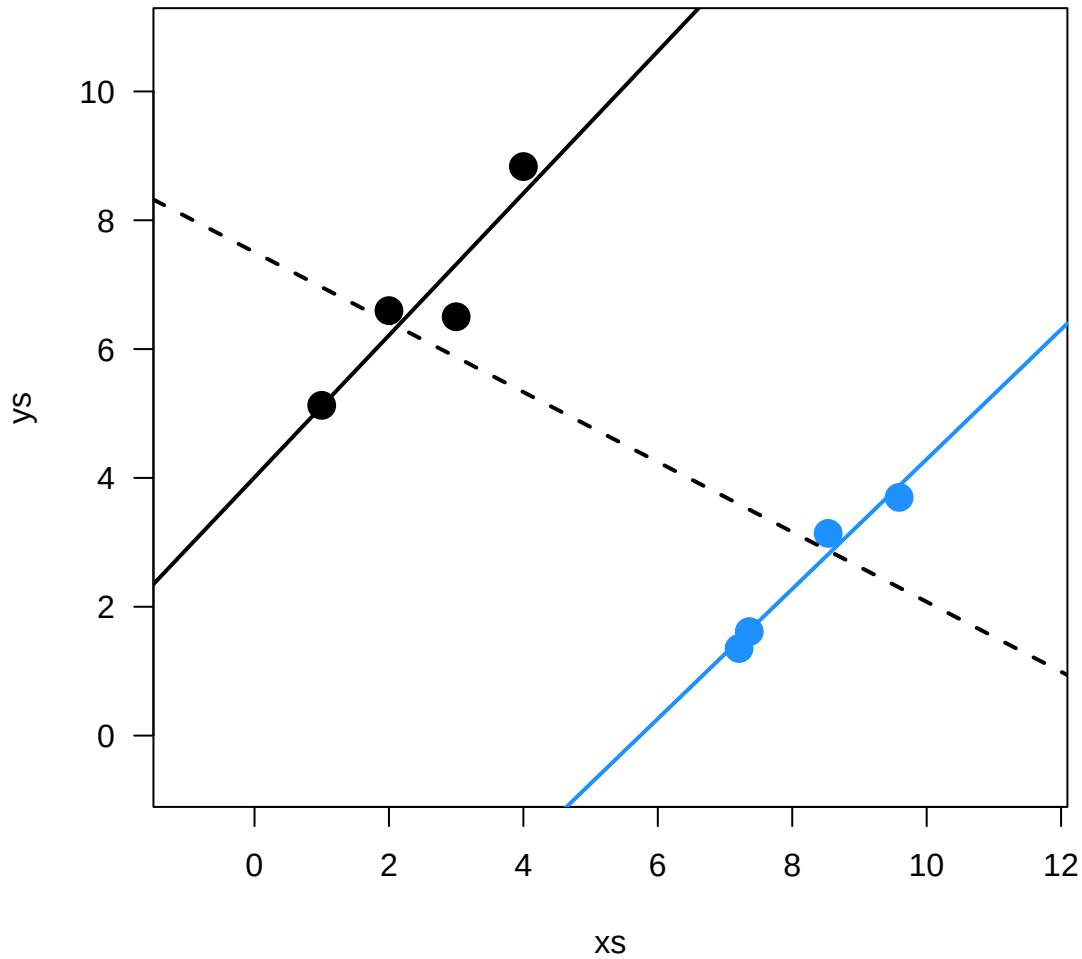
```
xs1 <- c(1,2,3,4)
ys1 <- xs1 + runif(4, 3, 5)

xs2 <- xs1 + runif(4, 5, 7)
ys2 <- xs2 - runif(4, 5, 7)

xs <- c(xs1,xs2)
ys <- c(ys1,ys2)

plot(xs,ys, cex=2, pch=16, las=1,
     col=rep(c(1, 'dodgerblue'), each=4),
     bg=rep(c("lightblue", "pink"), each=4),
     xlim=range(xs)+c(-2,2), ylim=range(ys)+c(-2,2))
abline(lm(ys1 ~ xs1), col=1, lwd=2)
```

```
abline(lm(ys2 ~ xs2), col='dodgerblue', lwd=2)
abline(lm(ys ~ xs), lwd=2, lty=2)
```



Measuring model error

Now that we have the ability to construct models, we need to be able to assess how well they are fitting. We can use the `summary()` function to get information on both individual predictor significance, as well as overall model performance. Overall model performance is assessed in `glm` using R-squared. This measure is bounded between 0 and 1, and corresponds to the proportion of the variance of our y that is explained by our predicts $x_1 \dots x_n$.

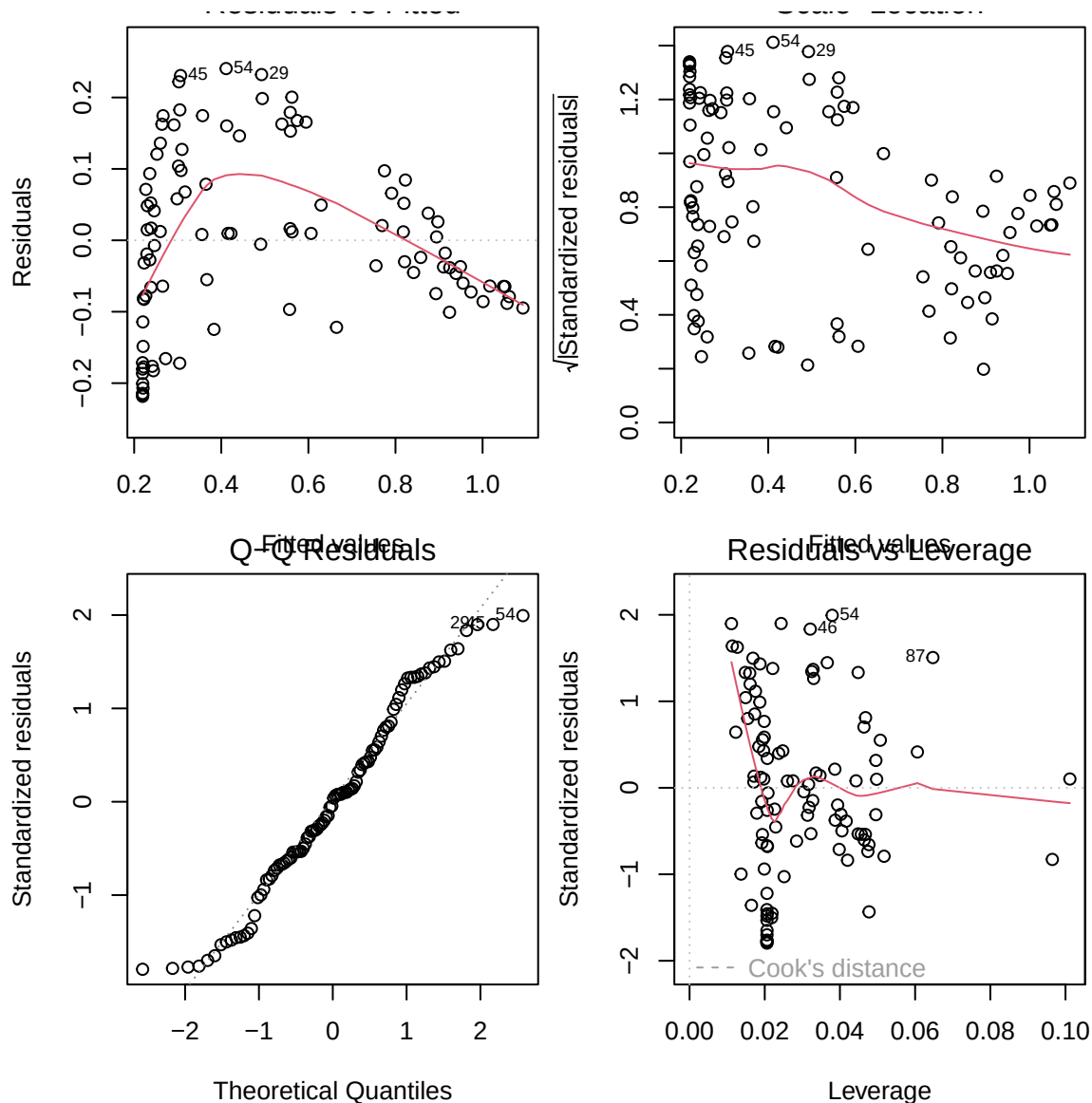
```
summary(m6)
```

```
##
## Call:
## lm(formula = y ~ x1 + x2, data = dat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.218455 -0.079515 -0.000387  0.086634  0.240474
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.21931    0.01764  12.430  < 2e-16 ***
```

```
## x1          0.83238    0.18342    4.538 1.63e-05 ***
## x2          0.04559    0.18372    0.248    0.805
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1229 on 97 degrees of freedom
## Multiple R-squared:  0.8502, Adjusted R-squared:  0.8472
## F-statistic: 275.4 on 2 and 97 DF,  p-value: < 2.2e-16
```

But R-squared only tells us so much. What we may want to look at is how the model is truly fitting. That is, are there areas of the predictor data that do not capture the variation in y equally? This situation would lead to a distribution of residual variation (model residuals are the differences between model-predicted values or fitted values from the true values). To explore that, we can use the base `plot` command on the model object to explore some of these things

```
layout(matrix(1:4, ncol=2))
par(mar=c(4,4,0.5,0.5))
plot(m6)
```



- Residuals vs fitted - there should be no strong patterns and no outliers, residuals should be randomly distributed around zero.
- Normal Q-Q - residuals should go around the diagonal line, i.e., should be normally distributed. This plot helps checking if they are approximately normal.
- Scale-location - as you can see, on Y axis there are also residuals (like in Residuals vs fitted plot), but they are scaled, so it is similar to (1), but in some cases it works better.
- Residuals vs Leverage – it helps to diagnose outlying cases. As in previous plots, outlying cases are numbered, but on this plot if there are any cases that are very different from the rest of the data they are plotted below thin red lines (check wiki on Cook's distance).

Model predictions

The measures of model performance discussed above are based on how well the predictors can explain the response, but, as we noted in the n -order polynomial case, a model could have an R-squared of 1 and be utterly useless. A non-useless model would arguably be able to predict to new data. That is, if more x data were added, the response y could be predicted. We can extract predictions from our model object using the predict function. We can do this for the entire data used to fit the models, or a new data object.

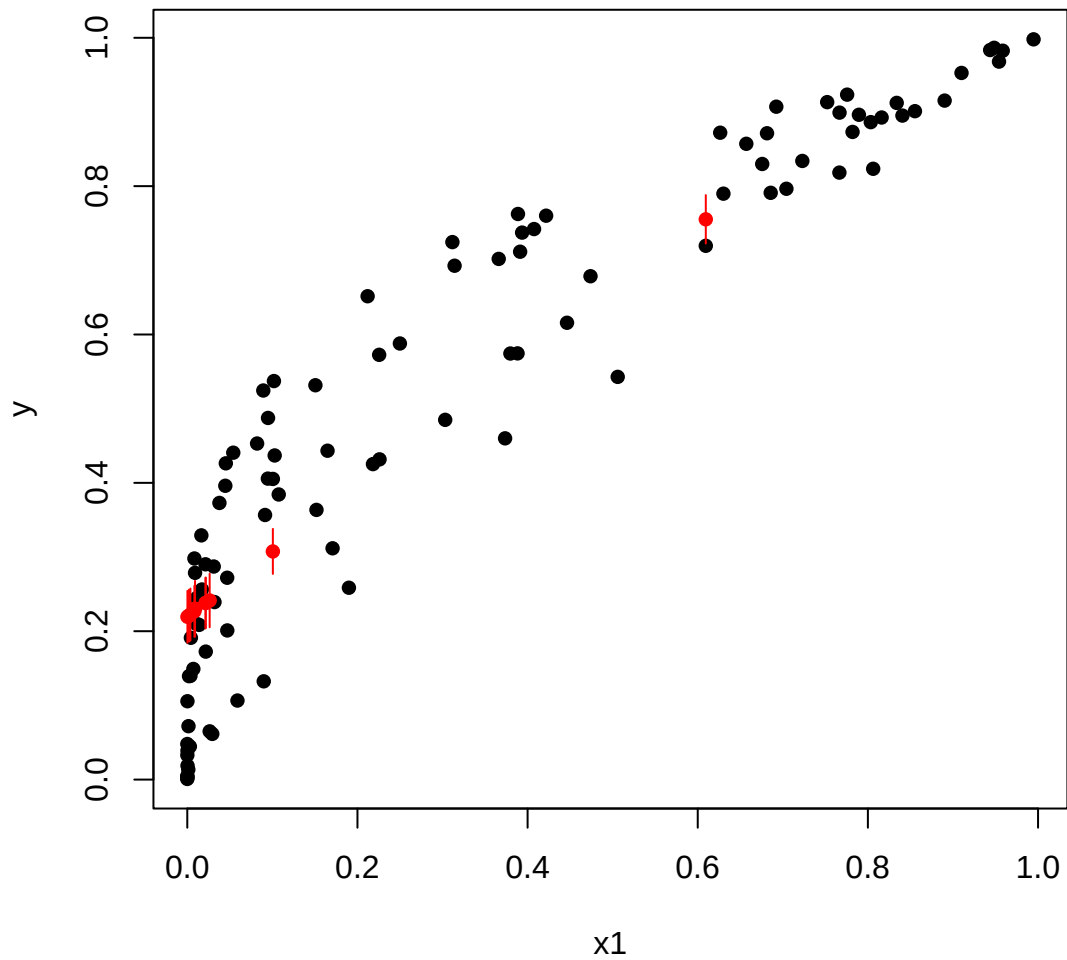
```
newData <- data.frame(x1=x1[1:10], x2=x2[1:10])

predict(m6, newData, interval='confidence')

##          fit      lwr      upr
## 1  0.2303482 0.1927556 0.2679407
## 2  0.3075525 0.2771907 0.3379144
## 3  0.2379210 0.2036765 0.2721655
## 4  0.2267578 0.1923580 0.2611576
## 5  0.2209987 0.1859349 0.2560625
## 6  0.7552656 0.7226210 0.7879101
## 7  0.2194798 0.1844752 0.2544845
## 8  0.2383320 0.2043589 0.2723050
## 9  0.2414341 0.2052893 0.2775788
## 10 0.2223970 0.1872763 0.2575176

preds <- as.data.frame(predict(m6, newData, interval='confidence'))

plot(y=y, x=x1, col=1, pch=16)
points(y=preds$fit, x=x1[1:10], pch=16, col='red')
segments(y0=preds$lwr, y1=preds$upr, x0=x1[1:10], col='red')
```

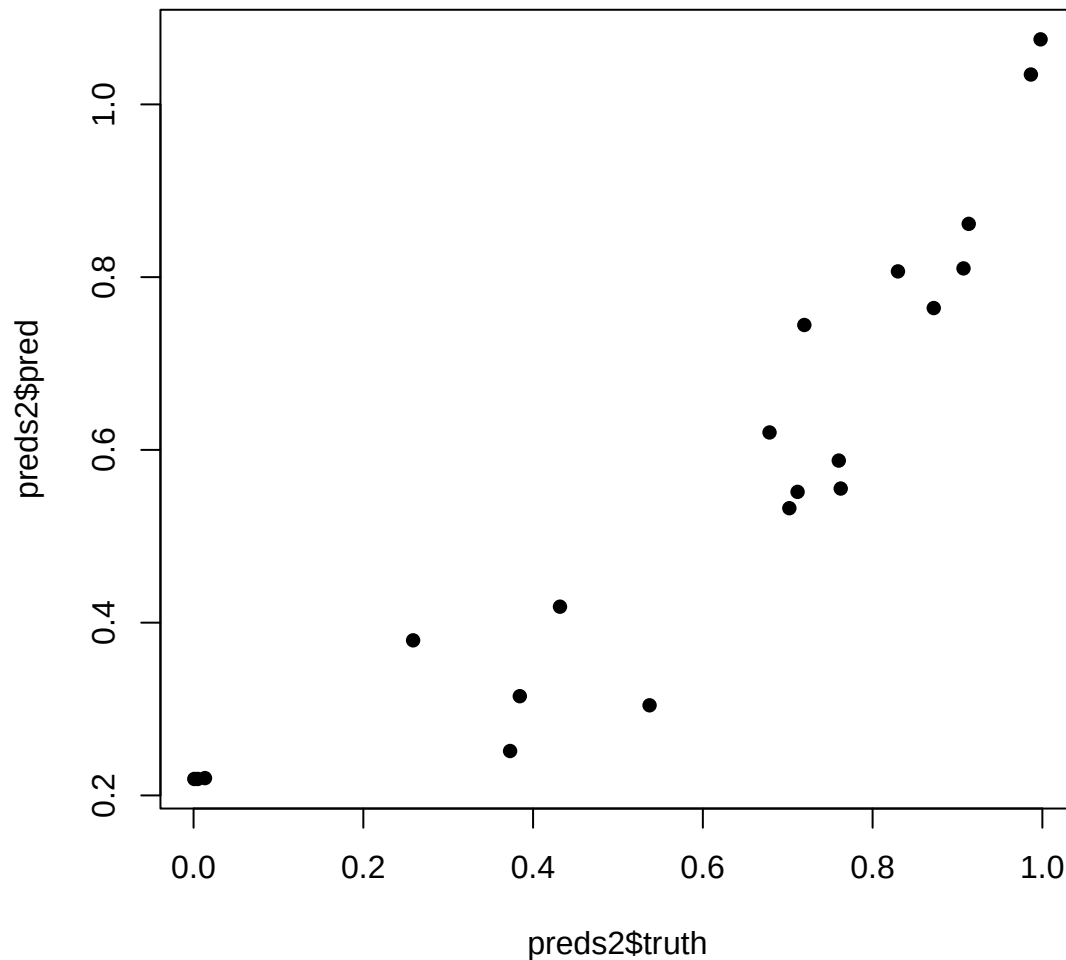


But we may wish to predict on new data or on a held out subset of data not used to train our models. This way, we can assess the performance of the model independent of the trained model conditioned on the existing data.

```
trainMod <- function(dat, holdOut=0.2){
  inds <- sample(1:nrow(dat), round(nrow(dat)*holdOut,0))
  train <- dat[-inds, ]
  test <- dat[inds, ]
  mod <- glm(y ~ x1+x2, data=train)
  ret <- data.frame(truth=test$y, pred=predict(mod, newdata=test))
  return(ret)
}

preds2 <- trainMod(dat)

plot(preds2$truth, preds2$pred,
     pch=16)
```



Machine learning

This idea of assessing model performance on a holdout set is not something traditional in ecological model building. This could potentially be that ecologists want to use *all* of their data to try to get at which variables are the most associated or explain the most variance in some response variable. This is fair. But the model will always be conditioned upon on the data available (I will not go into Bayesian modeling here). The book I recommend above sort of lays a lot of the foundation down for learning more about machine learning. Machine learning methods are typically designed to bypass some of the assumptions of more traditional models like those described above. Often, this results in methods that handle NAs well, are flexible to non-linear relationships between response and predictor variables, and that tend to be pretty focused on prediction over inference (this is not inherently true though). In fact, the book on statistical learning spends the majority of time going over the inner workings of linear models for regression and classification. It isn't until about halfway through the book where they introduce more “machine learning -esque” methods for classification and regression such as support vector machines and tree-based approaches.

```
# install.packages('gbm')

trainMod2 <- function(dat, holdOut=0.2){
  inds <- sample(1:nrow(dat), round(nrow(dat)*holdOut,0))
  train <- dat[-inds, ]
  test <- dat[inds, ]
  mod <- glm(y ~ x1+x2, data=train)
  mod2 <- gbm::gbm(y ~ x1+x2, data=train, n.trees=100,
    interaction.depth=2, cv.folds=5, n.cores=2)
```

```
brt.best.iter <- gbm::gbm.perf(mod2, method='OOB', plot=FALSE)
ret <- data.frame(truth=test$y,
  predGLM=predict(mod, newdata=test),
  predGBM=predict(mod2, newdata=test, type='response', n.trees=brt.best.iter))
return(ret)
}
```

```
preds3 <- trainMod2(dat)
```

```
## Distribution not specified, assuming gaussian ...
```

```
## OOB generally underestimates the optimal number of iterations although predictive performance is rea
```

```
glmAcc <- sqrt(mean((preds3$truth - preds3$predGLM)**2))
```

```
gbmAcc <- sqrt(mean((preds3$truth - preds3$predGBM)**2))
```

It is important to note that the model accuracy will change as it uses a random subset of 20% holdout data each time the function is called. We can use that to our advantage when comparing models, as we can train the model n times to see if the distributions of accuracy are distinct between models. It is also important to note here though that we can have lots of control over the outcome of any statistical test for model differences here by increasing the number of times we train the models to minimize the error in the distribution of accuracy values.

```
glmA <- gbmA <- c()
for(i in 1:10){
  set.seed(i)
  preds3 <- trainMod2(dat)

  glmA[i] <- sqrt(mean((preds3$truth - preds3$predGLM)**2))
  gbmA[i] <- sqrt(mean((preds3$truth - preds3$predGBM)**2))
}
```

And we can use a t-test to see if the distributions of accuracy values are different between the glm and the gbm.

```
t.test(glmA, gbmA)
```

```
##
## Welch Two Sample t-test
##
## data: glmA and gbmA
## t = 2.8523, df = 17.84, p-value = 0.01065
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
## 0.004117258 0.027199534
## sample estimates:
## mean of x mean of y
## 0.1225847 0.1069263
```

On average, using the boosted regression tree resulted in a change in root mean squared error of $\text{mean}(\text{gbmA} - \text{glmA})$ when training 100 models. What happens if we train 1000 models?

```
glmA2 <- gbmA2 <- c()
for(i in 1:100){
  set.seed(i)
  preds3 <- trainMod2(dat)
```



```

glmA2[i] <- sqrt(mean((preds3$truth - preds3$predGLM)**2))
gbmA2[i] <- sqrt(mean((preds3$truth - preds3$predGBM)**2))
}

```

This may not really influence the mean difference between the models in terms of root mean squared errors ($\text{mean}(\text{gbmA2} - \text{glmA2})$), but it will influence the resulting statistical test that we inappropriately apply to prove a point.

```

t.test(glmA2, gbmA2)

##
## Welch Two Sample t-test
##
## data: glmA2 and gbmA2
## t = 9.0171, df = 194.06, p-value < 2.2e-16
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##  0.01416995 0.02210392
## sample estimates:
## mean of x mean of y
## 0.1246660 0.1065291

```

Species distribution modeling

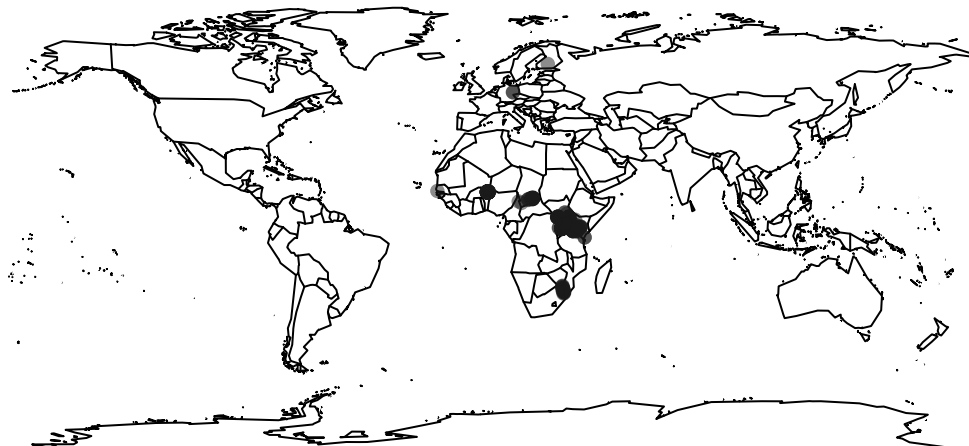
Recall from last lecture on querying APIs, where we pulled occurrence records for red pandas from GBIF? We may want to develop predictive models of habitat suitability, in order to estimate where red pandas could potentially occur. These models are often called 'species distribution models', and are essentially just a form of logistic regression. That is, we have binary data, in that species occurrences are either known (1), or we have no data, so we can sample points randomly and treat them as absences (0). These are generally called pseudo-absences, but the point of them is not to treat them as absences per se, but to try to get at the distribution of your predictor values relative to the known occurrences (**raise your hand if that makes no sense**).

```

giraffe <- rgbif::occ_search(scientificName = "Giraffa camelopardalis",
                             limit = 1000)[[3]]

maps::map()
points(giraffe$decimalLongitude, giraffe$decimalLatitude, pch=16,
       col=grey(0.1,0.5))

```



Alright. So we have the positive occurrence records for giraffes. To build a species distribution model, we need background points as well as environmental data.

```
bc <- geodata::worldclim_global(var="bio", res=10, path='.')

bcN <- terra::spatSample(bc, 5000, 'random')
bcN <- bcN[which(!is.na(bcN[,1])), ]
bcN <- bcN[1:1000, ]
bcN$prez <- 0

bcP <- terra::extract(bc, giraffe[,which(colnames(giraffe) %in% c('decimalLatitude', 'decimalLongitude'))])
bcP <- bcP[which(!is.na(bcP[,2])), ]
bcP$prez <- 1
bcP$ID <- NULL

dat <- rbind(bcN, bcP)
```

So we used the geodata package to get the bioclim data, extracted data from these raster layers for each sampled point (both background and presence points). So we are ready to construct a model. We can start with the simplest model, a logistic regression. This assumes a linear response to environmental conditions, where the binary data are transformed into a probit space.

```
logModel <- glm(prez ~ ., data=dat, family='binomial')

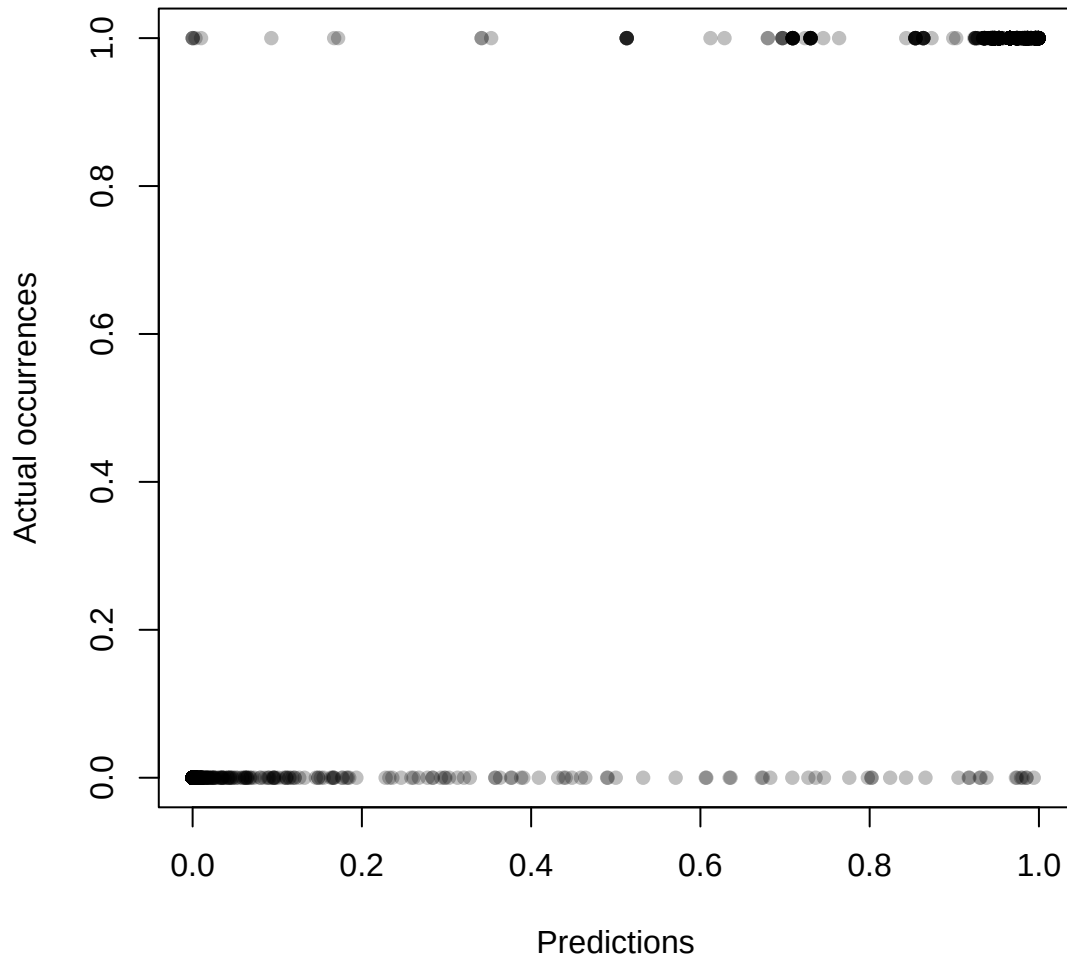
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
summary(logModel)

##
## Call:
## glm(formula = prez ~ ., family = "binomial", data = dat)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -1.702e+01  3.738e+00 -4.554 5.27e-06 ***
## wc2.1_10m_bio_1 -5.102e+00  7.864e-01 -6.488 8.70e-11 ***
## wc2.1_10m_bio_2 -1.492e+00  4.383e-01 -3.404 0.000664 ***
## wc2.1_10m_bio_3 -1.461e-01  5.985e-02 -2.441 0.014653 *
## wc2.1_10m_bio_4  2.465e-01  3.227e-02  7.639 2.20e-14 ***
## wc2.1_10m_bio_5  4.606e+04  1.677e+05  0.275 0.783553
## wc2.1_10m_bio_6 -4.606e+04  1.677e+05 -0.275 0.783557
## wc2.1_10m_bio_7 -4.606e+04  1.677e+05 -0.275 0.783565
## wc2.1_10m_bio_8  5.077e-02  6.302e-02  0.806 0.420526
## wc2.1_10m_bio_9  1.854e-01  7.058e-02  2.627 0.008622 **
## wc2.1_10m_bio_10 -1.011e+01  1.350e+00 -7.486 7.07e-14 ***
## wc2.1_10m_bio_11  1.479e+01  1.660e+00  8.912 < 2e-16 ***
## wc2.1_10m_bio_12  2.739e-02  4.792e-03  5.716 1.09e-08 ***
## wc2.1_10m_bio_13  2.447e-01  3.292e-02  7.433 1.06e-13 ***
## wc2.1_10m_bio_14  2.467e-01  9.468e-02  2.605 0.009180 **
## wc2.1_10m_bio_15 -3.036e-02  8.087e-03 -3.755 0.000174 ***
## wc2.1_10m_bio_16 -1.714e-01  2.023e-02 -8.472 < 2e-16 ***
## wc2.1_10m_bio_17 -1.259e-01  3.364e-02 -3.743 0.000182 ***
## wc2.1_10m_bio_18  1.450e-02  5.194e-03  2.792 0.005239 **
## wc2.1_10m_bio_19  1.116e-02  3.588e-03  3.110 0.001870 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
## (Dispersion parameter for binomial family taken to be 1)
##
## Null deviance: 2437.04 on 1777 degrees of freedom
## Residual deviance: 374.32 on 1758 degrees of freedom
## AIC: 414.32
##
## Number of Fisher Scoring iterations: 10
```

This indicates that many of the bioclim variables have a significant effect on occurrence, but it does not really give us an idea of how well the model performs (we will go over more details of model performance very soon). In the meantime, we can at least visualize the predicted values versus the true values. We will discuss why this is not a great measure of performance soon though.

```
plot(predict(logModel, type='response'), dat$prez,
     pch=16, col=adjustcolor(1,0.25), xlab='Predictions',
     ylab='Actual occurrences')
```



sessionInfo

```
sessionInfo()
```

```
## R version 4.5.0 (2025-04-11)
## Platform: x86_64-pc-linux-gnu
```

```

## Running under: Ubuntu 22.04.5 LTS
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/atlas/libblas.so.3.10.3
## LAPACK: /usr/lib/x86_64-linux-gnu/atlas/liblapack.so.3.10.3; LAPACK version 3.10.0
##
## locale:
## [1] LC_CTYPE=en_US.UTF-8      LC_NUMERIC=C
## [3] LC_TIME=en_US.UTF-8      LC_COLLATE=en_US.UTF-8
## [5] LC_MONETARY=en_US.UTF-8  LC_MESSAGES=en_US.UTF-8
## [7] LC_PAPER=en_US.UTF-8     LC_NAME=C
## [9] LC_ADDRESS=C             LC_TELEPHONE=C
## [11] LC_MEASUREMENT=en_US.UTF-8 LC_IDENTIFICATION=C
##
## time zone: America/New_York
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods    base
##
## other attached packages:
## [1] maps_3.4.1      gbm_2.2.2      igraph_2.1.4   tidyr_1.3.1    dplyr_1.1.4
## [6] plyr_1.8.9      DBI_1.2.3      geodata_0.5-8  terra_1.8-50   sf_1.0-21
## [11] rgbif_3.7.7     jsonlite_2.0.0 httr_1.4.7
##
## loaded via a namespace (and not attached):
## [1] gtable_0.3.6      xfun_0.52       ggplot2_3.5.2    lattice_0.22-5
## [5] vctr_0.6.5        tools_4.5.0     generics_0.1.4   parallel_4.5.0
## [9] curl_6.3.0        tibble_3.3.0    proxy_0.4-27     RSQLite_2.3.7
## [13] blob_1.2.4        pkgconfig_2.0.3 Matrix_1.7-3     KernSmooth_2.23-26
## [17] data.table_1.17.6 dbplyr_2.5.0     RColorBrewer_1.1-3 lifecycle_1.0.4
## [21] compiler_4.5.0    farver_2.1.2    stringr_1.5.1    tinytex_0.57
## [25] codetools_0.2-19 htmltools_0.5.8.1 class_7.3-23     yaml_2.3.10
## [29] lazyeval_0.2.2    pillar_1.10.2   whisker_0.4.1    classInt_0.4-11
## [33] cachem_1.1.0      viridis_0.6.5   tidymodels_1.2.1 digest_0.6.37
## [37] stringi_1.8.7     purrr_1.0.4     splines_4.5.0    fastmap_1.2.0
## [41] grid_4.5.0        cli_3.6.5       magrittr_2.0.3    survival_3.8-3
## [45] utf8_1.2.6        triebeard_0.4.1 crul_1.5.0        e1071_1.7-16
## [49] withr_3.0.2       scales_1.4.0    bit64_4.6.0-1    oai_0.4.0
## [53] rmarkdown_2.29    bit_4.6.0       gridExtra_2.3     memoise_2.0.1
## [57] evaluate_1.0.4    knitr_1.50      viridisLite_0.4.2 rlang_1.1.6
## [61] urltools_1.7.3    Rcpp_1.0.14     glue_1.8.0        httpcode_0.3.0
## [65] xml2_1.3.8        R6_2.6.1        units_0.8-7

```